HIGH-LEVEL DESIGN DOCUMENT

Agentic Customer Support Assistant

A production architecture for long-running, multi-agent systems — built on the OpenAI Agents SDK, Claude (via LiteLLM), durable workflows, streaming, and human-in-the-loop approvals. Explained in plain language.

OpenAI Agents SDK | Claude / LiteLLM | Temporal (durable) | FastAPI + SSE | Redis Streams | Agentic Memory | HITL

Use case: a support assistant that can work a single ticket for **days**

Audience: engineers & architects · Reading level: friendly, low mental load

Contents

1 · The big idea

We are building a **customer support assistant** that doesn't just answer one question and stop. It can **investigate, take actions, ask a human for approval, wait days for an outside event, and then finish the job** — all on its own, while keeping the customer updated live.

> **IN PLAIN TERMS**
>
> A normal chatbot is like a vending machine: one question in, one answer out, done. An **agentic** system is like a capable employee: it has a goal, it decides what steps to take, it uses tools (look up an order, issue a refund), it knows when to ask a manager for sign-off, and it can pick up a task again tomorrow where it left off.

What makes it "agentic"

- **It decides, step by step.** An LLM (Claude) chooses the next action each turn, instead of following a fixed script.
- **It uses tools.** Searching policy docs, looking up an order, issuing a refund — each is a tool the agent can call.
- **It is a team, not a soloist.** A supervisor delegates to managers, who delegate to workers. Each is focused on one job.
- **It is durable.** If the server restarts, or it's waiting three days for a shipping company, the work is **not lost**.
- **It keeps a human in charge of risky steps.** Refunds get approved by a person before they happen.

2 · The use case — a 3-day support ticket

We'll follow **one realistic ticket** through the whole document so every component has a concrete meaning.

> **THE TICKET**
> **Maria** writes in: "My order #1234 never arrived, and I think I was charged twice. Please help." She's a premium member who prefers email.

Why this needs an agentic system

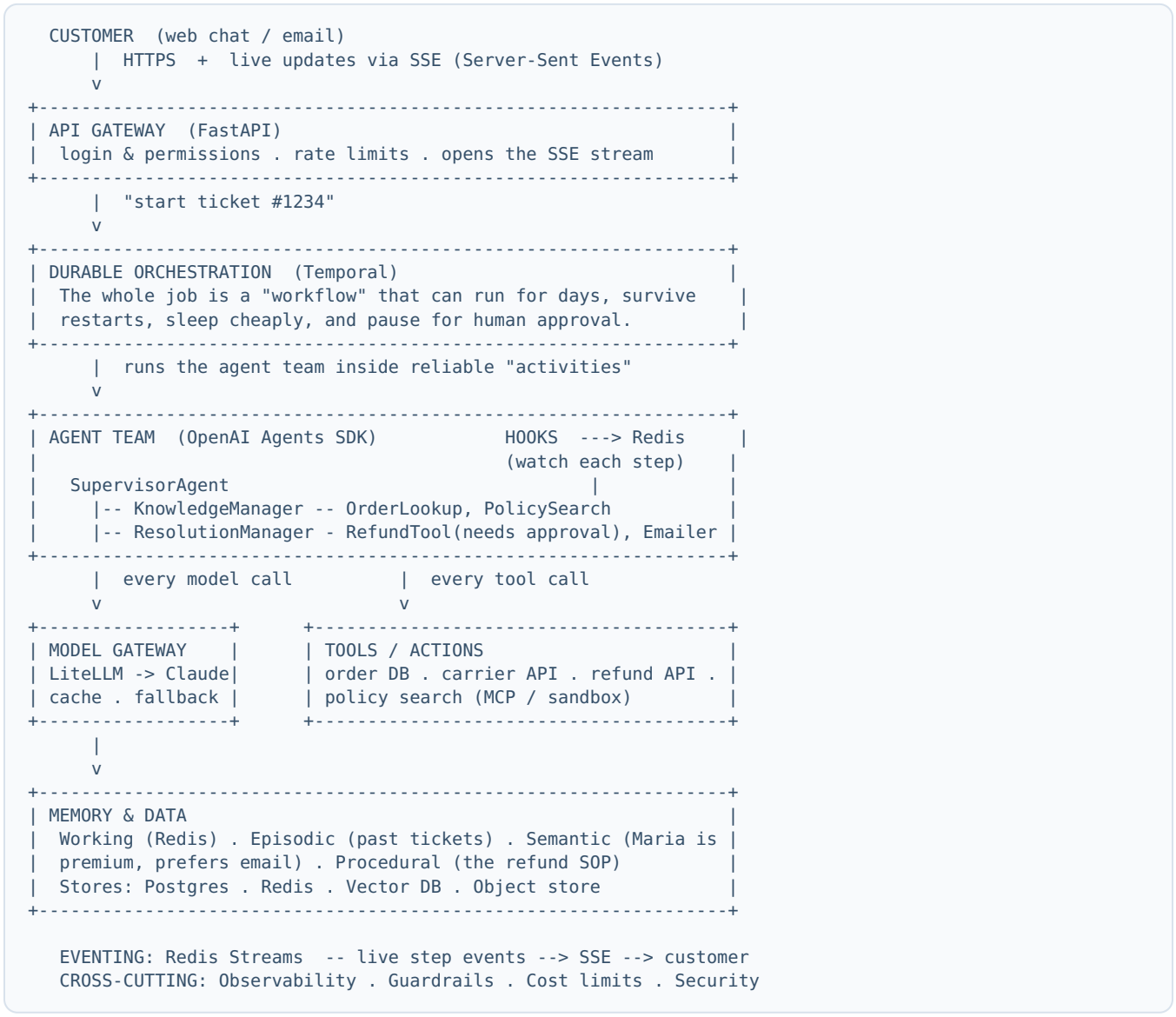
Resolving this isn't one step — it's a small project:

1. **Understand** — read the message, pull Maria's account & order history, check the refund and shipping policies.
2. **Act** — open a "lost package" claim with the carrier, draft a refund for the double charge, write a friendly reply.
3. **Get approval** — a refund over $50 needs a **human agent's sign-off** (this might take a few hours until someone is free).
4. **Wait** — the carrier's "lost package" investigation can take **2–3 days**. The assistant goes to sleep and wakes up when the carrier responds.
5. **Finish** — once the carrier confirms the loss, reship or refund, email Maria, and close the ticket.

> **THE LONG-RUNNING PART**
> Steps 3 and 4 mean a single ticket can stay "in progress" for **days**. The assistant must survive deploys, restarts, and long waits — without holding a server open or losing its place. This is the heart of the design.

3 · Architecture at a glance

Everything in this document fits in one picture. Don't worry about the details yet — we'll explain each box in plain terms.



HOW TO READ IT TOP-TO-BOTTOM

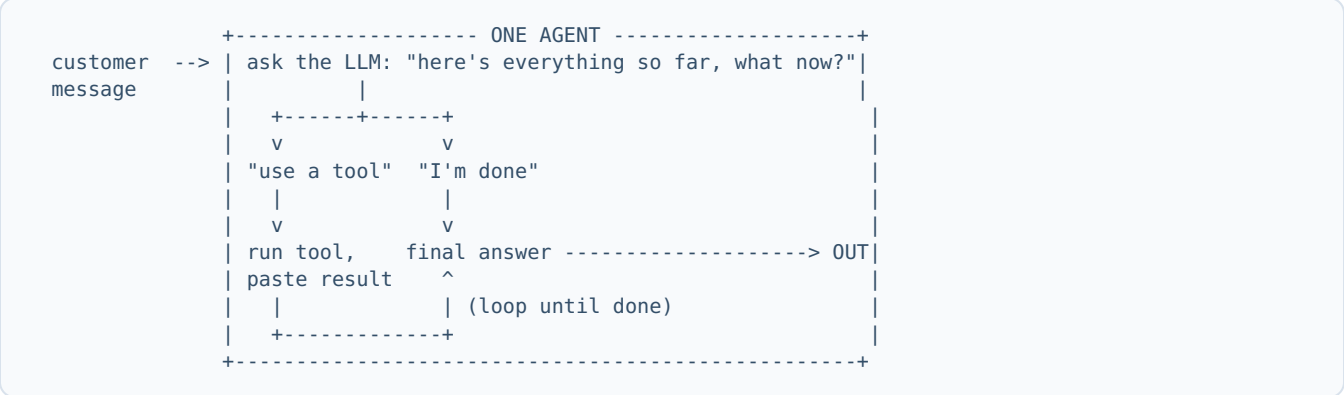
The customer talks to a **front door** (FastAPI). The front door starts a **durable job** (Temporal). The job runs the **agent team** (OpenAI Agents SDK). The agents call **Claude** to think and **tools** to act. Everything they learn is saved in **memory**. As they work, **hooks** push live updates through **Redis** back to the customer's screen.

4 · Mental model — agents are loops, the system is a tree

An agent is a loop around an LLM

An "agent" is not magic. It is a **loop** wrapped around one LLM. Each lap is called a **turn**. On every turn the LLM gets the whole story so far and does one of two things:

- **"Use a tool first"** → it names a tool (e.g. look up order #1234). The system runs the tool, pastes the result back in, and loops again.
- **"I'm done"** → it writes a final answer. The loop ends.



WHAT STOPS THE LOOP RUNNING FOREVER — "MAX TURNS"

Because the LLM decides when to stop, a confused model could loop forever (search → "let me search again" → search → ...). **max_turns** is a hard ceiling — a lap counter. Before each new turn the loop checks "have I hit the limit?" If yes, it **stops and raises an error** rather than looping again. It's the seatbelt: no matter how confused the model gets, the loop cannot run more than N times.

The system is a tree of agents

A tool can itself be **another whole agent**. So a manager's "tools" are its workers, and the supervisor's "tools" are its managers. Our support assistant:

```
SupervisorAgent              "understand the ticket, then resolve it"
  |
  |-- KnowledgeManager       "find out what happened"
  |     |-- OrderLookup      tool: read the order database
  |     |-- PolicySearch     tool: search refund / shipping policy docs
  |
  |-- ResolutionManager      "fix it"
        |-- RefundTool       tool: issue refund   (NEEDS HUMAN APPROVAL)
        |-- CarrierClaim     tool: open lost-package claim with carrier
        |-- Emailer          tool: draft & send the reply to Maria
```

In the current codebase the two managers are `ResearchManagerAgent` and `BuildManagerAgent`; for support we simply rename and re-tool them. The shape is identical.

5 · The Agent Graph — how `AgentTree` & `AgentNode` build it

There are **two** related things, and it helps to keep them separate:

Thing	What it is	Job
The executable graph	Real `Agent` objects from the OpenAI Agents SDK, wired together with `agent.as_tool(...)`.	Actually runs the work.
The AgentTree	A lightweight description of the same hierarchy — names, tools, parent/child links.	Describes the work: drives the sidebar diagram and lets the UI look up any node by path.

AgentNode — one box in the tree

An **AgentNode** is a single agent's entry in the map. It holds just three things:

- **name** — e.g. `"KnowledgeManager"`
- **tools** — a list of tool names it can use (`add_tool("OrderLookup")`)
- **children** — the agents beneath it (`add_child(order_lookup_node)`)

> **IN PLAIN TERMS**
>
> Think of an org chart. An **AgentNode** is one person's box: their name, the tools on their desk, and the people who report to them. **AgentTree** is the whole chart — it starts from the boss (the Supervisor) and links every box together.

AgentTree — wiring the boxes into a chart

The tree is built once, bottom-up: create each node, attach its tools, then `add_child` to hang workers under managers and managers under the supervisor. The result is a single root (Supervisor) you can walk downward.

```
build the chart:
   root      = AgentNode("SupervisorAgent")
   knowledge = AgentNode("KnowledgeManager")
   order     = AgentNode("OrderLookup");  order.add_tool("order_db")
   policy    = AgentNode("PolicySearch"); policy.add_tool("policy_kb")
   knowledge.add_child(order)
   knowledge.add_child(policy)
   root.add_child(knowledge)        ...and so on for ResolutionManager
   -> AgentTree(root)               one connected hierarchy
```

Path lookup. Because every node has a name and children, you can find any node by a dotted path like `SupervisorAgent.ResolutionManager.RefundTool` — the tree walks name-by-name. This is how the UI highlights "where are we right now" and how the tool path in the State Inspector is resolved.

> **TECH & FILES**
>
> `orchestration/agent_tree.py` → `AgentNode` (name, tools, children, `find` by path, `to_dict`) and `AgentTree.build_default_tree()` . The executable graph is built separately in `src/agents/`

> `factory.py` → `build_supervisor()`, which creates SDK `Agent`s and wires children via `traced_as_tool()` (a thin wrapper over `agent.as_tool`). The AgentTree mirrors that graph for display.

Agent-as-tool vs handoff (why we chose as-tool)

	Agent-as-tool (we use this)	Handoff (not used)
Analogy	Ask a contractor, they return a result, **you keep the project**.	Transfer the phone call; **the other person now owns it**.
Control after	**Returns** to the caller	**Transfers** away
Call several & combine?	Yes	No (one-way)
Who writes the final answer	The caller (it aggregates)	The agent handed to

Our supervisor must call **both** managers and **combine** their results into one reply for Maria — that's "call & return", so we use **agent-as-tool**. Handoff would let a manager become the final responder, which can't aggregate.

6 · Component deep-dives

6.1 · Clients & the API Gateway (FastAPI)

> **IN PLAIN TERMS**
>
> The API Gateway is the **front door** of the building. Everyone enters here. It checks your ID (login), makes sure you're not knocking too fast (rate limits), and shows you to the right room (your tenant/company's data).

What it does: authentication & permissions, rate limiting, request validation, multi-tenant isolation, and — importantly — it **opens the live stream** back to the customer (see 6.2). It is a thin, fast layer; it does no thinking itself.

> **IN OUR SUPPORT ASSISTANT**
>
> Maria's chat widget calls the gateway. The gateway confirms she's logged in, mints a `request_id` for this ticket, starts the durable job, and opens an SSE stream so her screen can show progress live.

> **TECH**
>
> FastAPI (Python). Today the repo uses Streamlit as the only entry point; in production FastAPI becomes the API, and Streamlit (or a React app) is just one client.

6.2 · Streaming — how the customer sees progress live

A long job is boring if the screen just spins. We **stream** progress so Maria sees "Looking up your order… Checking the refund policy… Drafting your reply…" as it happens. Three ideas make this work.

(a) What is a FastAPI streaming response?

> **IN PLAIN TERMS**
>
> Normally a web server cooks the whole meal and brings one full plate. A **streaming response** brings food out dish by dish as it's ready. The connection stays open and each piece is sent the moment it exists.

Technically, instead of returning a finished string, the endpoint returns a `StreamingResponse` wrapping an **async generator**. Each time the generator `yield`s a chunk, FastAPI flushes it down the open connection. The browser receives bytes continuously rather than waiting for the end.

(b) What is SSE (Server-Sent Events)?

> **IN PLAIN TERMS**

SSE is a **one-way radio broadcast** from server to browser over a single, long-lived HTTP connection. The server keeps sending little text messages; the browser listens and reacts. It's simpler than WebSockets (which are two-way) and perfect when only the server needs to push.

The format is just text: the server sends lines like `data: {...}\n\n`. The browser uses the built-in `EventSource` object to receive each event and update the screen. The response's content type is `text/event-stream`.

(c) How do the agents use it?

The agents don't talk to the browser directly. The flow is a relay:

```
agent does a step (or Claude emits a token)
      |
      v
HOOK fires  ----------------->  publishes an event to REDIS STREAM
(on_tool_start, etc.)           key = ticket request_id
      :                                 |
      :                                 v
      :                    FastAPI SSE endpoint is "tailing" that stream
      :                                 |  yields:  data: {"step":"Looking up order"}\n\n
      :                                 v
      :                          browser EventSource --> updates Maria's screen
```

IN OUR SUPPORT ASSISTANT

When the `OrderLookup` tool starts, a hook publishes `{"step":"Looking up order #1234"}` to Redis. The SSE endpoint forwards it, and Maria instantly sees that line appear — even though the full resolution is still minutes (or days) away.

TECH

FastAPI `StreamingResponse` with `media_type="text/event-stream"` ; Redis Streams as the message bus; the browser's `EventSource` API. Token-level streaming comes from the model gateway; step-level events come from the hooks (next section).

6.3 · The OpenAI Agents SDK runtime — the agent loop

IN PLAIN TERMS

The OpenAI Agents SDK is the **engine** that runs the loop from Section 4 for us. We just describe each agent (its instructions and tools); the SDK handles calling the model, parsing which tool it wants, running that tool, feeding the result back, and looping until done.

What the SDK gives us (so we don't hand-write it):

- **The loop** (`Runner.run`) — model → tools → repeat → final answer.
- **Tools** — decorate a Python function and the SDK exposes it to the model.
- **Agent-as-tool** — turn an agent into a callable tool for its parent.
- **Shared context** — a "scratchpad" object passed to every agent in the run.
- **Hooks** — callbacks on every lifecycle event (start/end of agent, tool, model).
- **Pause/resume** — built-in support for stopping for human approval.

- **tool_use_behavior** — e.g. an agent can "stop right after the first tool" (our retrieval worker uses this so it doesn't waste a turn writing prose).

THE SCRATCHPAD (SHARED CONTEXT)

One blank "whiteboard" is created per ticket and passed into every agent. When `OrderLookup` finds the order details, it writes them on the whiteboard; `Emailer` later reads them off it. Only a short summary travels up the chain of command; the heavy materials travel sideways on the whiteboard. It's wiped when the ticket finishes — so tickets never leak into each other.

TECH & FILES

`openai-agents[litellm]` . In the repo: `src/agents/factory.py` (builds the agents), `sdk_tools.py` (the `RunContext` scratchpad + tools + `traced_as_tool`), and the bridge in `agent_defs/supervisor.py` that calls `Runner.run` .

6.4 · Hooks & the Redis streaming callback handler

IN PLAIN TERMS

A **hook** is a doorbell that rings on every event inside a run: "an agent started", "a tool finished", "the model replied". We attach a listener to those doorbells to do two things: **(1) push a live update** to the customer, and **(2) record metrics** (time, tokens, cost).

The SDK fires these lifecycle events; our hook object implements the matching methods:

Hook fires when…	We do…
`on_agent_start` / `on_agent_end`	count the agent, time it, emit "started/finished"
`on_tool_start` / `on_tool_end`	publish a live "Looking up order…" step; time the tool
`on_llm_start` / `on_llm_end`	record tokens & cost per agent

The "Redis streaming callback handler"

The hook doesn't know about browsers. It calls a small **callback handler** whose only job is "take this step and put it somewhere a stream can read it." In development that "somewhere" is an in-memory list (the Streamlit log). **In production it's a Redis Stream** — so the handler `publishes` each event to Redis, and the FastAPI SSE endpoint (6.2) tails it and forwards to the customer.

```
SDK lifecycle event --> Hook method --> callback handler --> Redis Stream --> SSE --> UI
     (on_tool_start)     (our code)      (push/publish)      (durable bus)   (live)
```

IN OUR SUPPORT ASSISTANT

Every visible line Maria sees ("Checking refund policy…", "Awaiting agent approval…") is a hook event that the Redis callback handler published. The same events double as the **audit log** of exactly what the assistant did, in order.

TECH & FILES

`src/agents/sdk_hooks.py` (`OrchestrationHooks` , a subclass of the SDK's `RunHooks`) + `orchestration/streaming_handler.py` (the callback handler with `push_orchestration_step`). Crucial detail: hooks are passed into **every** sub-run, because the SDK does not automatically pass them down to agent-as-tool children.

6.5 · Model gateway — LiteLLM → Claude

IN PLAIN TERMS

The model gateway is a **universal power adapter** for AI models. Our agents speak one language; the gateway plugs that into Claude (or any model) and handles the messy parts: retries, fallbacks if a model is down, caching repeated questions, and counting the bill.

- **Routing & fallback** — send to Claude; if it's unavailable, fail over.
- **Semantic cache** — if we already answered a near-identical question, reuse it (saves money & time).
- **Cost metering** — count tokens and dollars per customer/tenant.
- **Key management** — one safe place for API keys.

TECH

LiteLLM. Today it runs in-process (`LitellmModel` → `anthropic/claude-opus-4-8`). In production it becomes a separate **LiteLLM proxy** service so all apps share routing, caching and budgets. The model is **Claude (Opus 4.8)**.

6.6 · Tools / Actions — MCP & sandboxed execution

IN PLAIN TERMS

Tools are the assistant's **hands**. Thinking (Claude) decides what to do; tools actually do it — read the order database, call the carrier's API, issue the refund.

- **MCP servers** (Model Context Protocol) — a standard "plug" so tools from anywhere (the order system, the CRM) connect the same way, without custom glue each time.
- **Sandboxed execution** — if a tool runs code or touches money, run it in a locked room (a sandbox) so a mistake can't damage the wider system.

IN OUR SUPPORT ASSISTANT

`OrderLookup` (read-only), `CarrierClaim` (calls an external API), `RefundTool` (moves money → guarded by human approval + sandbox), `PolicySearch` (RAG over policy docs).

6.7 · Memory — the four kinds

IN PLAIN TERMS

> Memory is not one thing. People have a notepad for the task in hand, a diary of past events, a set of facts they know, and skills they've learned. Agents need all four.

Kind	Holds (in plain terms)	For Maria's ticket
Working (short-term)	the notepad for this task — the scratchpad	the current ticket: order details just looked up
Episodic	a diary of what happened before	"Maria had a similar late delivery in March"
Semantic	durable facts & preferences	"Maria is premium and prefers email"
Procedural	learned how-to / standard procedures	the official refund SOP the assistant follows

The services that keep memory healthy: summarize long chats so they fit; reflect — a background pass that reads recent tickets and promotes durable facts into semantic memory; retrieve the most relevant memories (by relevance + recency + importance); and forget stale ones.

> **TECH**
> Redis (working), a Vector DB like pgvector/Qdrant (episodic & semantic search), a Graph DB like Neo4j (relationships), an object store for big artifacts. Frameworks like Mem0 / Letta / Zep can provide this off the shelf. Today the repo has only working memory (the scratchpad) and a toy keyword search — the rest is the production add.

6.8 · Durable orchestration — Temporal & long-running agents

> **IN PLAIN TERMS**
> Temporal is a **save-game system** for long jobs. After every step it saves progress. If the power goes out (a crash, a deploy), it reloads the save and continues from exactly where it stopped. It can also "sleep" for days at no cost and wake on cue.

Why we need it for a 3-day ticket

The agent loop normally lives inside a running process. If that process restarts while Maria's ticket is waiting on the carrier, the work would vanish. Temporal moves the **state out of the process** so nothing is lost.

Agent idea	Temporal piece	Plain meaning
the whole ticket job	**Workflow**	the durable, resumable script
one model or tool call	**Activity**	a single retryable step
"wait for agent approval"	**Signal**	pause for days at zero cost; resume on a message
"check carrier again in 2 days"	**Timer**	a durable alarm clock

IN OUR SUPPORT ASSISTANT

Maria's ticket is one Temporal **Workflow**. Looking up the order is an **Activity**. Waiting for the human refund approval is a **Signal**. Waiting for the carrier's 2–3 day investigation is a **Timer**. If the servers are redeployed on day 2, the workflow simply resumes — Maria never notices.

TECH

Temporal. It has a first-party integration with the OpenAI Agents SDK, so the agent graph runs inside workflows with little rewrite. (Alternatives: Restate, DBOS, AWS Step Functions.) The repo's HITL `RunState` JSON is a small-scale version of the same idea; Temporal generalizes it.

6.9 · Eventing — Redis Streams

IN PLAIN TERMS

Redis Streams is a **conveyor belt of events**. Anyone can drop an event on the belt; many readers can pick events off it. We use it to carry live updates and to record a tidy log of everything that happened.

- **Live updates** → feed the SSE stream to the customer (6.2).
- **Audit / replay** → every step is on the belt, so we can rebuild the timeline.
- **Work queues** → hand tasks to a pool of workers, with a "dead-letter" lane for poison messages.

Temporal handles durable orchestration; Redis Streams handles real-time eventing. They are partners, not substitutes. (Swap in Kafka for very high scale.)

6.10 · Data stores & Observability

Store	Holds
Postgres	tickets, users, audit, structured state
Redis	working memory, cache, locks, the event streams
Vector DB	embeddings for episodic/semantic memory & policy search
Object store (S3)	large artifacts (attachments, generated documents)

OBSERVABILITY — IN PLAIN TERMS

A **flight recorder**. For every ticket it records a timeline (traces), counters (metrics like cost & tokens per agent), and logs. When something is slow or wrong, you can see exactly which agent or tool caused it.

TECH

OpenTelemetry traces → Tempo; Prometheus + Grafana metrics; structured logs; optional LLM-trace UIs (Langfuse/Phoenix). **Already built into the repo** (per-agent cost/tokens), gated by `OBS_ENABLED`.

7 · Human-in-the-loop (HITL)

Some steps are too risky to let the AI do alone — issuing a refund, for example. **HITL** means the system **pauses**, asks a human to approve, and only then continues. Here's exactly how that works.

> **IN PLAIN TERMS**
>
> It's like a junior employee who can do most things, but needs a manager's signature for anything involving money. They prepare everything, then stop and wait at the manager's desk. When the signature comes, they carry on. If the manager says no, they take a different route.

How a pause happens (the effect on the graph)

We mark the risky step — the `RefundTool` (and the manager that calls it) — as **"needs approval"**. When the supervisor's loop reaches that tool call, the SDK does **not** run it. Instead it **stops the whole run** and reports an **interruption** (it says: "I want to call RefundTool with these arguments — approve?").

```
Supervisor loop ... reaches "call ResolutionManager -> RefundTool"
      |
      |   tool is marked needs_approval
      v
 RUN PAUSES HERE  (interruption raised)
      |          the refund has NOT happened; the sub-run did NOT execute
      v
 save the exact state to disk  ----> wait for a human
```

> **EFFECT ON THE GRAPH — IMPORTANT**
>
> The pause happens **before** the guarded step runs. So nothing risky has occurred yet — the refund sub-task is "frozen at the doorway". Everything before it (looking up the order, checking policy) already ran and its results are safe on the scratchpad.

How the state is stored

At the pause, the SDK can serialize the **entire run** — the conversation, which step we're on, the pending tool call, and the scratchpad — into a **RunState** (a JSON blob). We wrap that blob inside a small record (the customer, the ticket, the pending action) and **save it**:

- **Today:** a JSON file per paused run in `.hitl_states/<id>.json` (plus kept in memory).
- **Production:** a row in Postgres (and, with Temporal, the pause is just a workflow blocked on a **Signal** — no manual saving needed at all).

> **TECH & FILES**
>
> `orchestration/hitl_manager.py` stores/loads the paused state; the SDK `RunState` JSON lives inside the record's `pending_data`. The approval UI reads the pending action (which agent, what it wants to do) from that record.

How it resumes

When the human clicks **Approve** (maybe hours later):

1. Load the saved record and pull out the **RunState** JSON.
2. Rebuild the **same agent graph** (it's built deterministically, so it matches).
3. Restore the run from the JSON, **mark the pending tool call as approved**, and press play again.
4. The loop continues **exactly where it stopped** — now the RefundTool actually runs, the email goes out, the ticket moves on.

Other choices the human has:

Decision	What happens in the graph
Approve	the frozen step runs; the loop continues to the final answer
Reject	the step is refused; the supervisor's LLM is told "no" and picks a different path (e.g. escalate to a human agent instead of auto-refunding)
Revise	the run restarts with edited input (e.g. "refund $40, not $80")

IN OUR SUPPORT ASSISTANT

The assistant prepares Maria's $80 refund and pauses. A support agent sees the request in a queue, confirms the double-charge, and clicks **Approve** two hours later. The workflow resumes, the refund is issued, and Maria gets her confirmation email — all without the assistant "starting over".

8 · End-to-end walkthrough — the 3-day ticket

#	What happens	Which components
1	Maria sends the message; the front door authenticates her and opens a live stream.	FastAPI gateway · SSE
2	A durable job (workflow) starts for ticket #1234.	Temporal
3	Memory is loaded: her past tickets, "premium, prefers email", the refund SOP.	Memory (episodic/semantic/procedural)
4	Supervisor → KnowledgeManager: look up the order, search policy. Maria sees these steps live.	Agents SDK · Tools · Hooks → Redis → SSE
5	Supervisor → ResolutionManager: open a carrier claim, prepare an $80 refund.	Agents SDK · Carrier API
6	**PAUSE** — the refund needs approval. State is saved; Maria sees "awaiting review".	HITL · RunState / Signal
7	Two hours later a human approves. The job resumes; the refund is issued.	HITL resume
8	The job **sleeps** waiting for the carrier's lost-package result.	Temporal Timer (days)
9	(Servers are redeployed on day 2 — the job survives and keeps waiting.)	Temporal durability
10	Day 3: the carrier confirms the loss (webhook). The job wakes, reships, emails Maria, closes the ticket.	Temporal Signal · Emailer
11	The episode is saved to memory; a reflection pass notes "Maria had two late deliveries → flag account".	Memory write · reflection
12	Full timeline, cost and token usage per agent are visible in dashboards.	Observability

THE PAYOFF

One ticket spanned **three days**, a **human approval**, a **server redeploy**, and a **multi-day external wait** — and the assistant handled it as one continuous, resumable job, keeping Maria informed the whole time. That is what this architecture buys you.

9 · Tech stack summary

Layer	Technology	Role in one line
Client	Web/React or Streamlit	where the customer chats
API & streaming	**FastAPI** + SSE (EventSource)	front door + live progress
Durable workflows	**Temporal**	long-running, crash-proof jobs
Agent framework	**OpenAI Agents SDK**	runs the supervisor→manager→worker loop
Model access	**LiteLLM → Claude (Opus 4.8)**	the reasoning brain + routing/cache/cost
Tools	**MCP** servers + sandbox	the assistant's hands (order DB, carrier, refund)
Eventing	**Redis Streams**	live events + audit + queues
Memory	Redis + Vector DB (pgvector/Qdrant) + Graph (Neo4j)	working / episodic / semantic / procedural
Data	Postgres + Redis + Object store (S3)	durable state, cache, artifacts
Observability	OpenTelemetry + Prometheus/Grafana + Langfuse	traces, metrics, cost — the flight recorder
Cross-cutting	Guardrails · budgets · Vault · Kubernetes	safety, cost control, secrets, scale

10 · Glossary

Term	Plain-English meaning
Agent	an LLM running in a loop that can use tools to reach a goal
Turn	one lap of that loop (one "think, maybe act" cycle)
max_turns	the hard lap-limit that stops a runaway loop
Agent-as-tool	using one agent as a callable tool of another; control returns to the caller
Handoff	transferring control to another agent entirely (one-way)
AgentNode / AgentTree	the org-chart of agents — names, tools, who reports to whom
Scratchpad (RunContext)	a shared whiteboard per ticket; materials travel sideways on it
Hook	a doorbell that rings on each event (agent/tool/model start & end)
Callback handler	the listener that turns hook events into stream messages
StreamingResponse	a web reply sent piece-by-piece as it's produced
SSE	Server-Sent Events: a one-way server→browser live text stream
HITL	human-in-the-loop: pause for a person to approve a risky step
RunState	a saved snapshot of a paused run (so it can resume later)
Workflow / Activity (Temporal)	the durable job / one retryable step inside it
Signal / Timer (Temporal)	"wake on a message" / "wake at a time" — both survive restarts
MCP	a standard way to plug external tools into the agent
RAG	retrieval-augmented generation: fetch relevant docs, then answer using them

High-Level Design · Agentic Customer Support Assistant · Built on the OpenAI Agents SDK + Claude (LiteLLM), Temporal, FastAPI/SSE, Redis Streams, and agentic memory. This document describes the target production architecture; the current repository implements the agent graph, hooks, observability, and HITL, with the durable/streaming/memory layers as the production roadmap.